

Assignment 16

Apache Kafka — 39 Theoretical Questions | Data Engineering Interview Notes

Part 1 — Kafka Fundamentals

1. What is Apache Kafka?

Apache Kafka is a distributed event-streaming platform used to publish, store, and process streams of records. Producers write events; consumers read them independently.

2. Why is Kafka used in data engineering?

Kafka supports real-time ingestion, decouples producers and consumers, scales through partitions and brokers, supports fault tolerance through replication, allows replay while records are retained, and connects systems such as Spark and data lakes.

Typical data flow: Applications / databases → Kafka → Spark / data lake / analytics.

3. What are the main components of Kafka?

Producer (publishes), consumer (reads), topic (named stream), partition (ordered log), broker (server), cluster (group of brokers), offset (position), replication (copies), and consumer group (consumers sharing partition assignments).

4. What is a Kafka topic?

A named category or stream to which records are published, such as orders, payments, or customer_events.

5. What is a partition in Kafka?

An ordered, append-only sequence of records within a topic. Each record has an offset.

6. Why does Kafka use partitions?

To distribute data, enable parallel consumption, increase potential throughput, and scale beyond one partition's capacity. Ordering is guaranteed within a partition, not across the whole topic.

7. What is a broker?

A Kafka server that stores partition replicas and handles producer and consumer requests.

8. What is a Kafka cluster?

A group of Kafka brokers working together to store data, replicate partitions, and serve clients.

9. What is the difference between a topic and a partition?

A topic is a logical stream; a partition is one ordered log within that topic. For example, orders is a topic and partition 0 is one part of it.

10. Can a topic have multiple partitions?

Yes. A topic can have one or many partitions, such as orders-0, orders-1, and orders-2.

11. Can multiple consumers read from the same topic?

Yes. Consumers in the same group share partition work; consumers in different groups can independently read the same retained records.

Part 2 — Partitions, Replication, and Brokers

12. What happens when you increase the number of partitions?

It can increase parallelism and throughput and allow more active consumers. Existing records are not automatically redistributed, key mapping may change, and extra partitions add resource overhead.

13. What is replication in Kafka?

Maintaining copies of a partition on multiple brokers. One replica is the leader; follower replicas copy its data.

14. What is a replication factor?

The number of replicas maintained for each partition. RF=1 means one copy; RF=3 means three copies. Placing three replicas on separate brokers generally requires at least three brokers.

15. Why is replication important?

It supports fault tolerance and availability. If a leader fails, an eligible in-sync follower may become leader. Replication is not a backup and cannot guarantee zero data loss in every configuration.

16. What happens if a Kafka broker goes down?

If it hosts only followers, leaders elsewhere may continue. If it hosts leaders, affected partitions may pause while Kafka elects new leaders. Without an eligible replica, a partition may remain unavailable.

17. What are follower replicas?

Replicas that copy data from a partition leader. An eligible follower may become leader if the current leader fails.

18. What is the difference between Kafka and a traditional message queue?

Kafka uses a retained, partitioned log and offsets; multiple consumer groups can independently read retained records. Traditional queues often emphasize delivery and acknowledgement. This is a general comparison—products vary and may also support replay and durable storage.

Part 3 — Producers and Partition Selection

19. What is a Kafka producer?

An application or service that publishes records to Kafka topics.

20. How does a producer send data to Kafka?

It creates a record, selects a topic and partition, sends it to the appropriate broker, and receives an acknowledgement according to producer configuration.

Producer flow: Create record → select topic/partition → send to broker → append record and acknowledge.

21. How does a producer decide which partition receives a message?

It can use an explicitly specified partition, a key-based partitioner, or the configured default partitioning strategy when no key or partition is provided.

22. What is a partition key?

A record value used by the partitioning strategy to help choose a destination partition, such as `customer_101`.

23. Why is a key important?

It helps route related records to the same partition, supports per-key ordering, and keeps related data together.

24. What happens if the producer doesn't provide a key?

The default partitioner chooses a partition. Depending on client version and configuration, it may use sticky partitioning rather than strict record-by-record round-robin.

25. Will records with the same key always go to the same partition?

Generally, if partition count and partitioning strategy remain unchanged. Increasing partition count can change mapping; custom partitioners and explicit partition selection can also change behavior.

26. What is producer batching?

Collecting multiple records and sending them together. Settings such as `batch.size` and `linger.ms` influence batching. It can improve throughput and reduce requests, but waiting to form batches may add latency.

Part 4 — Consumers, Offsets, and Consumer Groups

27. What is a Kafka consumer?

An application that reads records from Kafka, such as a PySpark Structured Streaming job.

28. How does a consumer read messages?

It subscribes to a topic and reads assigned partitions. It tracks progress using offsets and can resume from committed offsets, subject to retention and reset settings.

29. What is an offset?

The position of a record within a specific partition. Offsets are partition-specific, not global IDs for a topic.

30. Why are offsets important?

They track reading progress, enable resuming after restarts, allow progress to be committed, and support replay of retained records.

31. What is `auto.offset.reset`? Explain `earliest` and `latest`.

It controls where a consumer starts when it has no valid committed offset. `earliest` starts at the earliest retained record; `latest` starts at the end and reads new records; `none` raises an error. It does not normally override a valid committed offset.

32. What is a consumer group?

A set of consumers cooperating to consume a topic. Kafka assigns partitions among them; a partition is assigned to at most one consumer in a group at a time.

33. Why do we use consumer groups?

To distribute work, scale consumption, rebalance assignments when members change, and track independent progress for each group.

34. Can two consumers in the same group consume the same partition simultaneously?

Under normal group assignment, no. A partition is assigned to at most one consumer in that group at a time.

35. Can consumers from different groups consume the same message?

Yes. Each group maintains its own offsets, so groups can independently read the same retained records.

36. What happens if a consumer crashes?

Kafka detects the departure and rebalances the group. Partitions may be assigned to other consumers, which resume from committed offsets. Depending on commit timing, records may be processed again.

37. What happens when a new consumer joins a group?

Kafka may rebalance and redistribute partition assignments. The exact assignment depends on the assignment strategy and group protocol.

38. What happens when consumers outnumber partitions?

Some consumers are idle for that topic. With 3 partitions and 5 consumers, at most 3 consumers can receive a partition; 2 are idle.

39. What happens when consumers are fewer than partitions?

Some consumers receive multiple partitions. With 6 partitions and 2 consumers, a balanced assignment may give each 3 partitions.

Quick Revision

Term	Remember it as
Topic	Named stream of records
Partition	Ordered log within a topic
Broker / cluster	Kafka server / group of brokers
Replication factor	Number of copies per partition
Producer / consumer	Writes records / reads records
Key / offset	Helps select partition / position in partition
Consumer group	Consumers sharing partition assignments
earliest / latest	Earliest retained record / end when no valid offset exists

Interview takeaway: Kafka distributes records using partitions, protects them through replication, and scales consumption using consumer groups. Producers publish records; consumers track progress with offsets.